

JOB LISTINGS ANALYTICS DASHBOARD

Using Python, Web Scraping, Data Analytics & Streamlit

Portfolio-Level Project Report

Prepared from the supplied project brief

1. Project Overview

This project develops an end-to-end Job Listings Analytics Dashboard. The workflow follows the supplied brief: collect job data through web scraping, clean and preprocess it, engineer analytical features, perform statistical analysis and exploratory data analysis, visualize the findings, and publish the results through an interactive Streamlit dashboard.

The recommended source in the brief is the Real Python Fake Jobs practice site. The site is a static learning resource designed for web-scraping practice, so its listings are suitable for demonstrating the pipeline but should not be presented as real employment vacancies.

The implementation includes Python functions, a JobScraper class, exception handling, Requests, BeautifulSoup, NumPy, Pandas, Matplotlib, Seaborn, scikit-learn and Streamlit.

2. Objectives

- Understand a complete data-analytics workflow.
- Collect job information using Requests and BeautifulSoup.
- Store extracted records in a Pandas DataFrame and CSV.
- Handle missing values and duplicates.
- Convert data types and standardize text.
- Create new analytical features.
- Demonstrate label encoding and one-hot encoding.
- Perform univariate, bivariate and multivariate EDA.
- Build an interactive Streamlit dashboard with filters, statistics, charts, business insights and download functionality.

The project brief also expects source code, raw/cleaned/processed datasets, a working dashboard, a project report, screenshots and a demonstration of scraping, cleaning, feature engineering, EDA, dashboard functionality and insights.

3. Data Collection

The scraper requests the source page, locates the ResultsContainer, iterates through job cards, extracts title, company, location, posting date and application URL, and optionally visits the application/detail page to collect the job description. A custom ScrapingError and try/except handling make network failures explicit.

Source: <https://realpython.github.io/fake-jobs/> . The source contains job cards with titles, companies, locations and application links; the individual job pages also expose descriptions. The supplied brief specifically recommends this website.

The bundled demonstration dataset contains 100 records, satisfying the brief requirement of at least 100 records.

4. Dataset Information

Required core columns are job title, company name, location, job description/category and application link. Additional enriched columns include salary, experience level, job type/work mode and analytical features.

Metric	Value
Records	100
Columns	27
Companies	99
Categories	7
Average minimum salary	\$67,810
Most common category	Other
Most common work mode	On-site

5. Data Cleaning & Preprocessing

The cleaning module standardizes column names, creates missing columns when necessary, trims text, fills missing text values, removes duplicate records, converts the posting date to a datetime type, and separates location into city and region. Missing job descriptions are replaced with an explicit placeholder so downstream analysis remains stable.

The pipeline saves `cleaned_jobs.csv` for the next phase. This directly addresses the brief requirements for missing-value handling, duplicate removal and data-type conversion.

6. Feature Engineering

The feature-engineering module creates job category, experience level, work mode, salary minimum and maximum, salary category, title word count, description length, description word count, Python keyword flag and technical keyword count.

Label encoding is demonstrated through scikit-learn LabelEncoder for job category. One-hot encoding is demonstrated with Pandas get_dummies for experience level and work mode. The result is stored as processed_jobs.csv.

7. Exploratory Data Analysis

The EDA stage includes the analysis types requested in the brief: histogram, count plot, bar plot, box plot, violin plot, scatter plot, crosstab/heatmap, pair plot and correlation analysis. Every visualization is intended to be accompanied by an observation in the final presentation or viva.

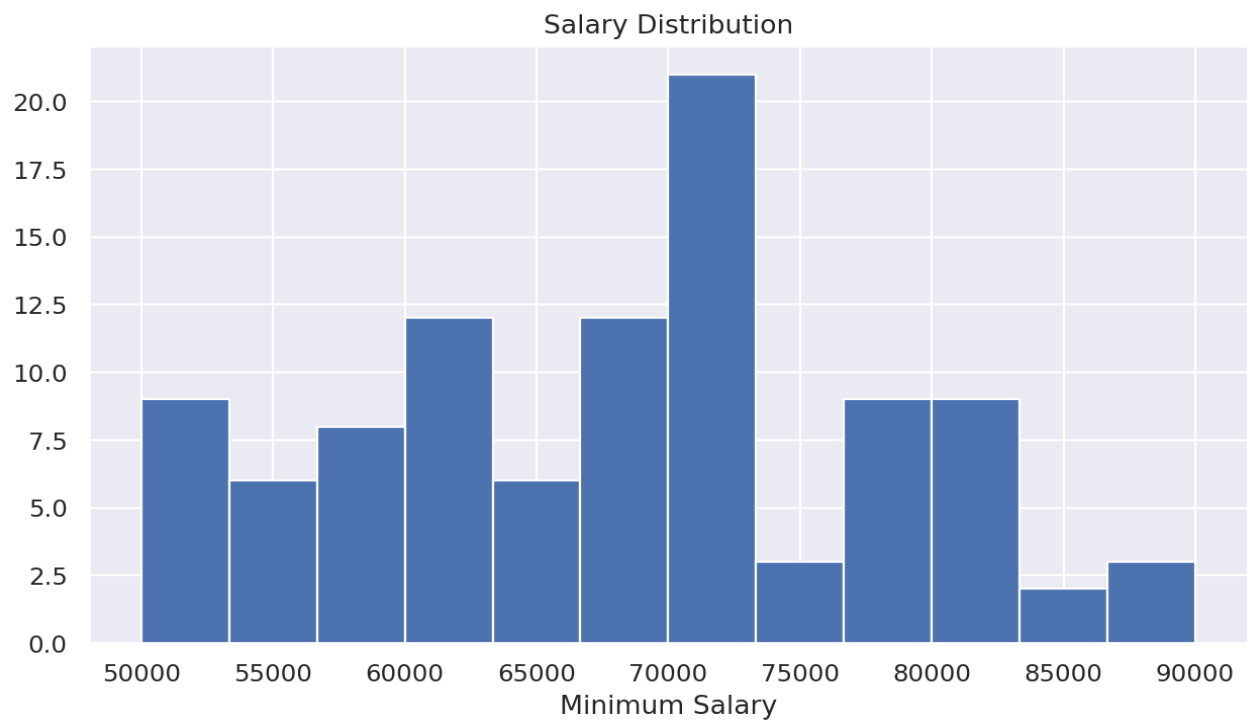


Figure 1. Salary distribution.

8. EDA Visualizations

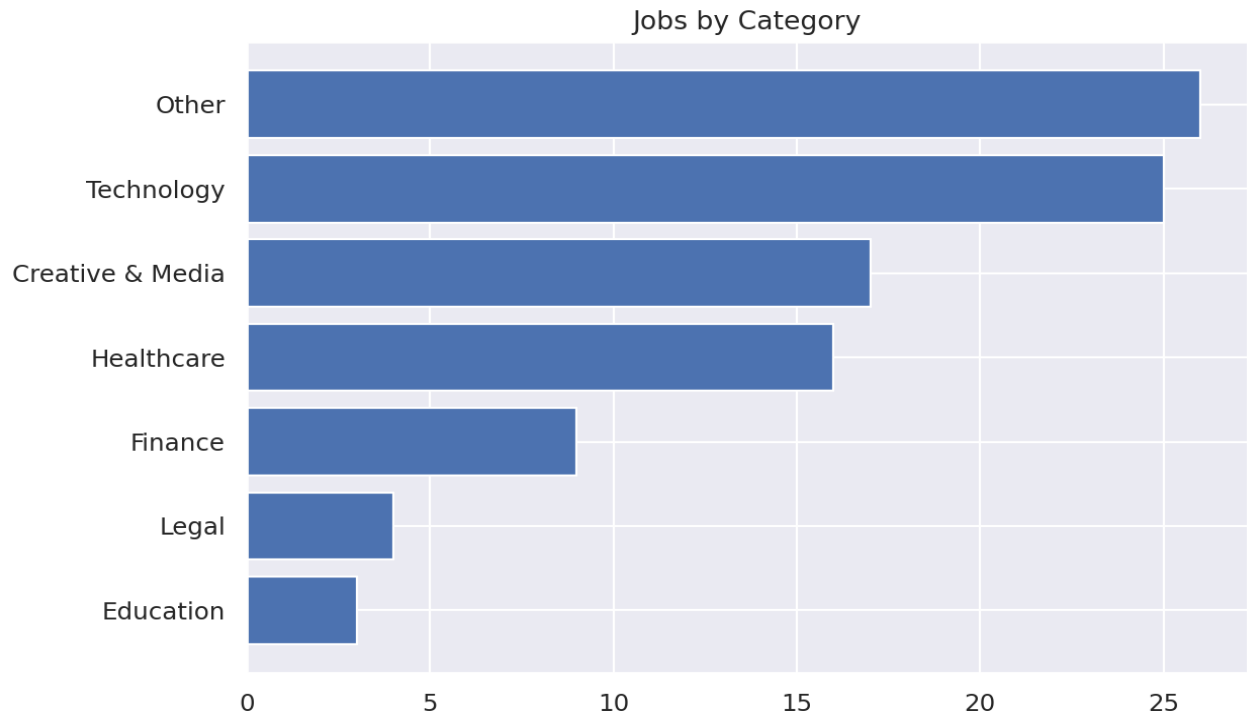


Figure 2. Job category distribution.

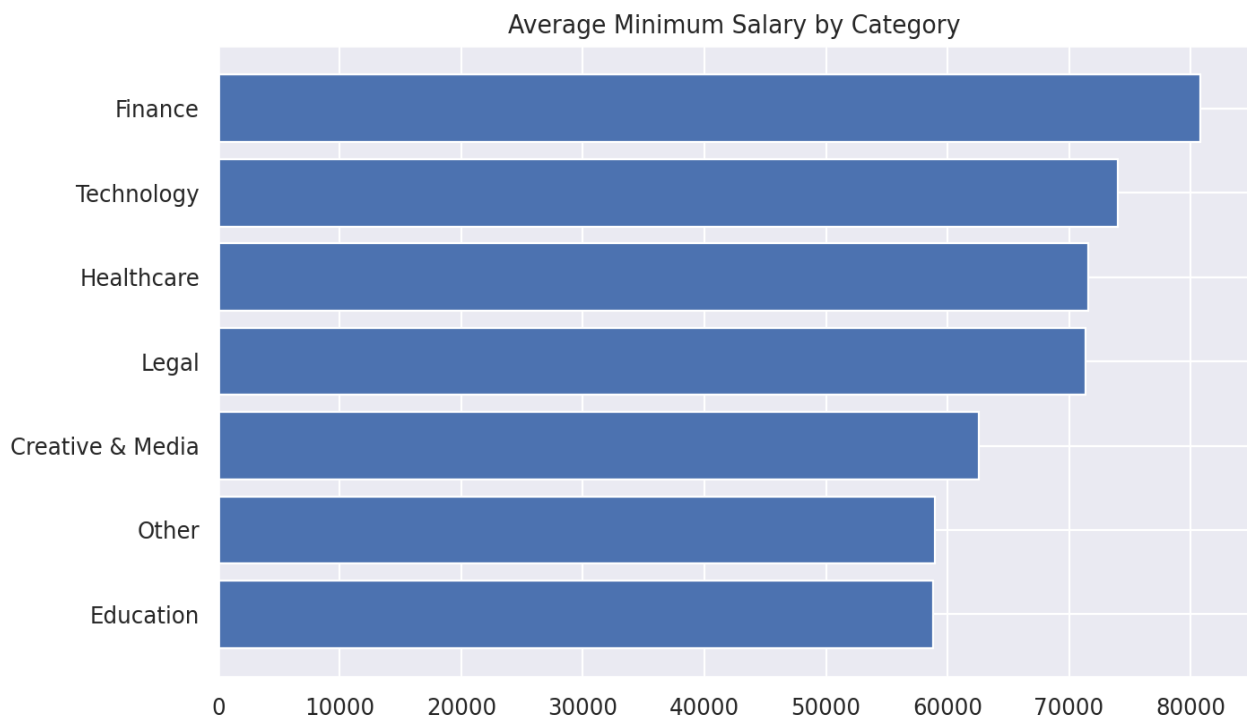


Figure 3. Average minimum salary by category.

9. Distribution & Relationship Analysis



Figure 4. Salary spread by category.

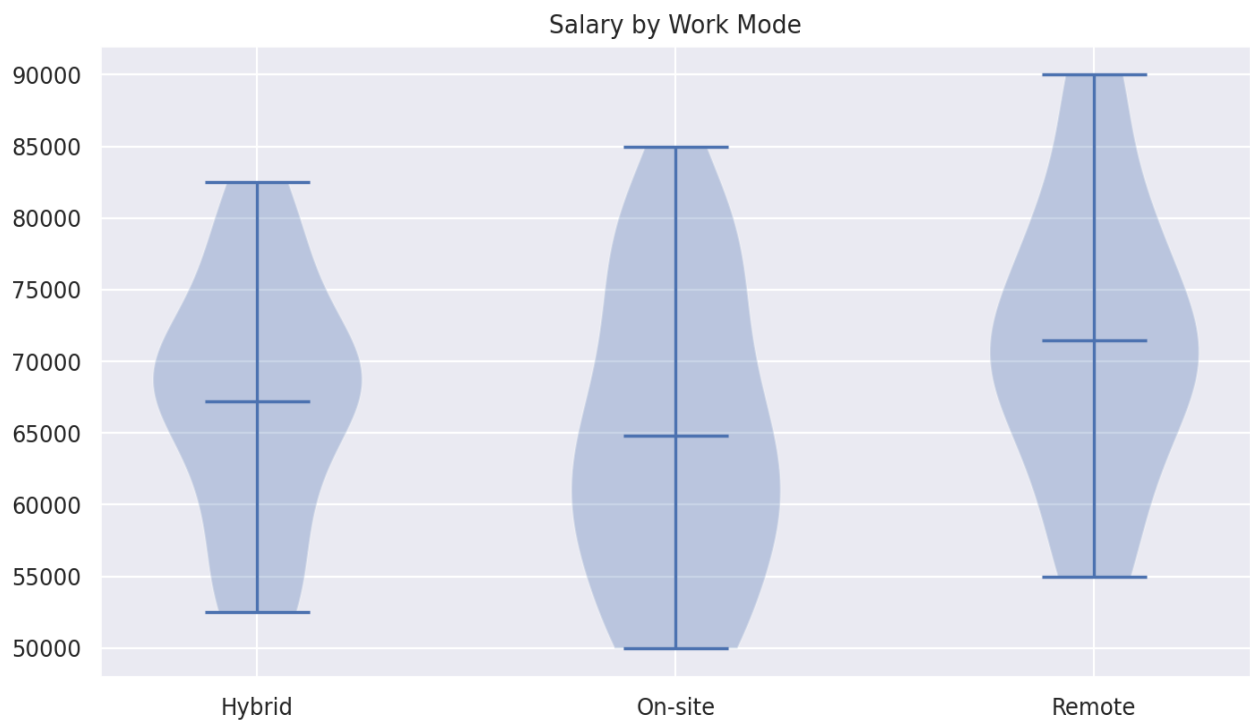


Figure 5. Salary distribution by work mode.

10. Multivariate Analysis

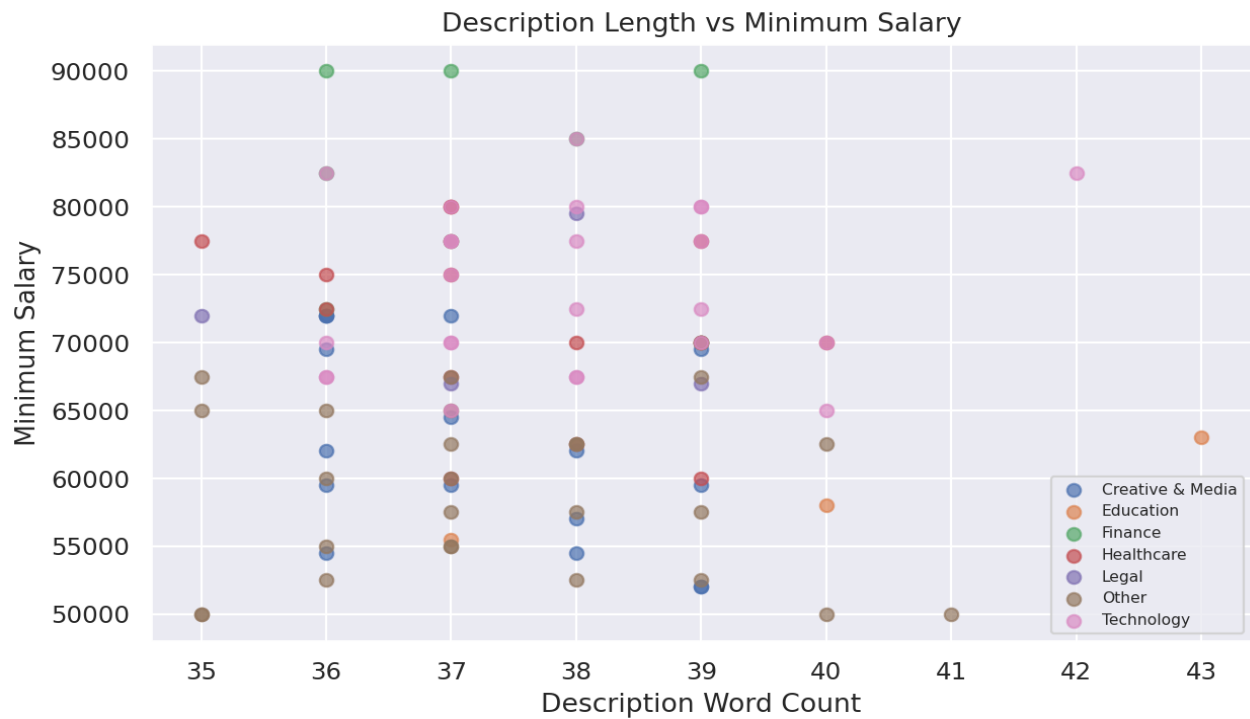


Figure 6. Description length versus minimum salary.

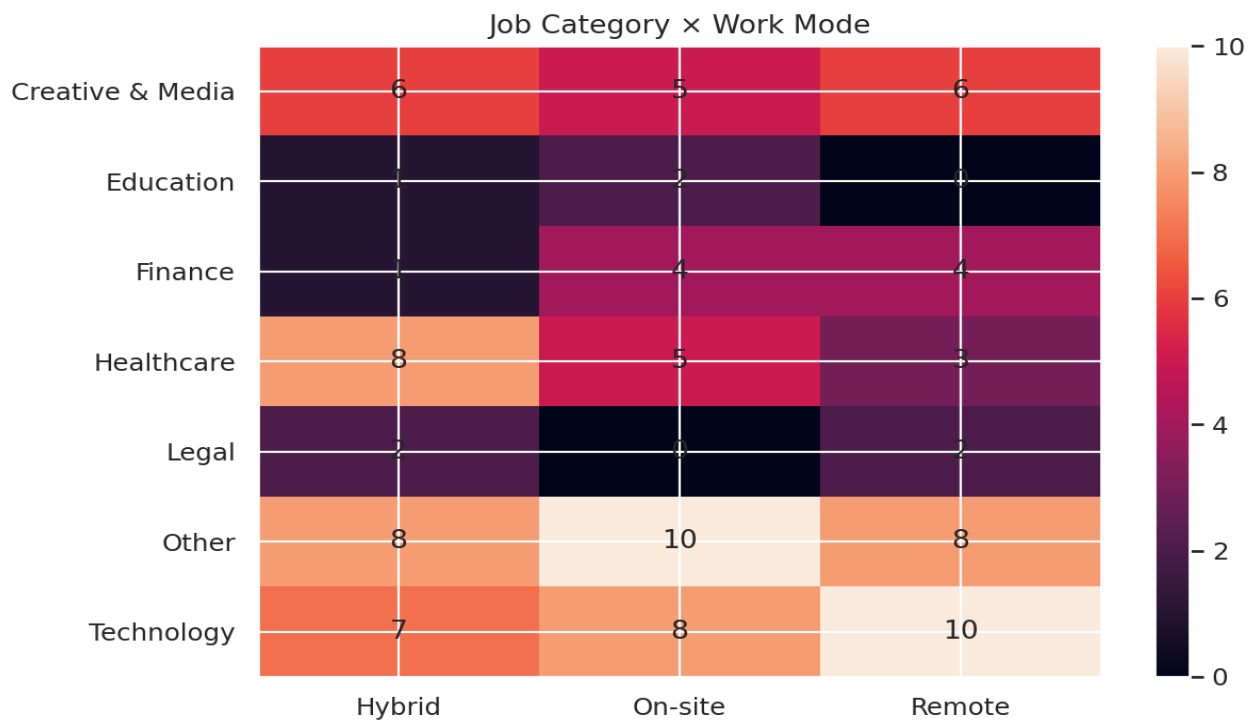


Figure 7. Job category and work-mode crosstab.

11. Correlation & Pair Analysis

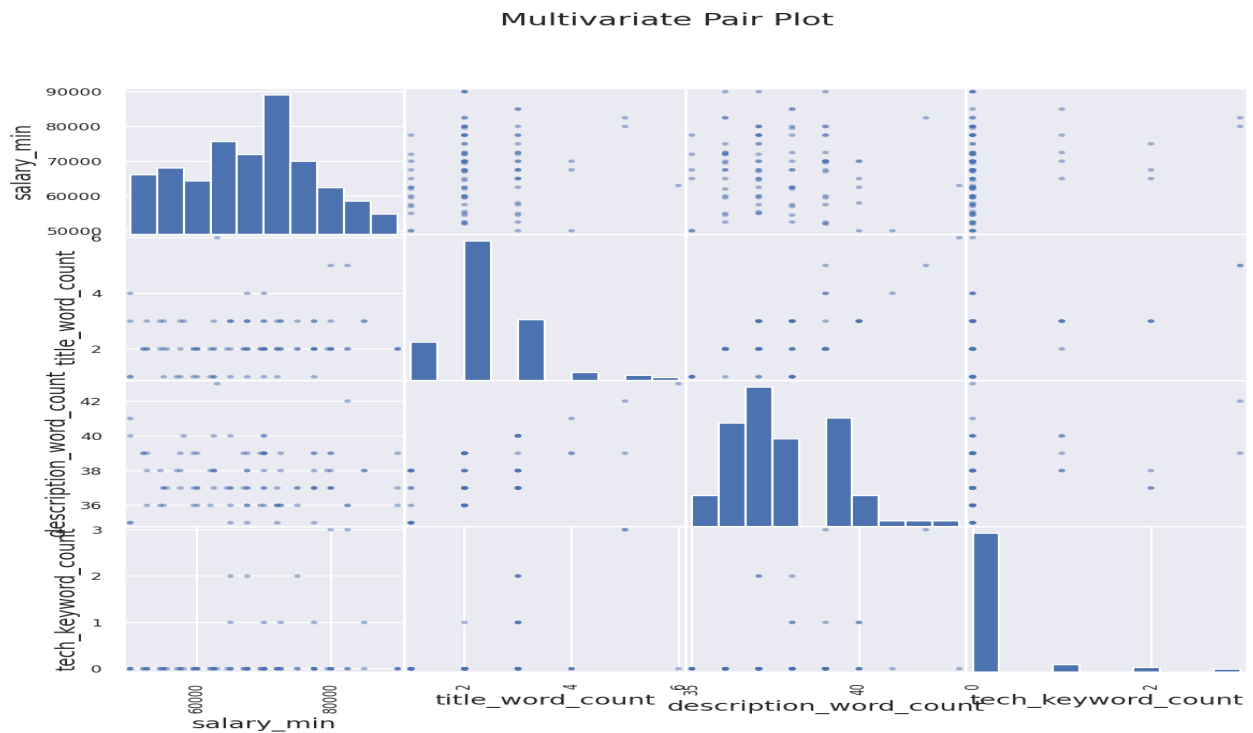


Figure 8. Pair plot of selected numerical features.

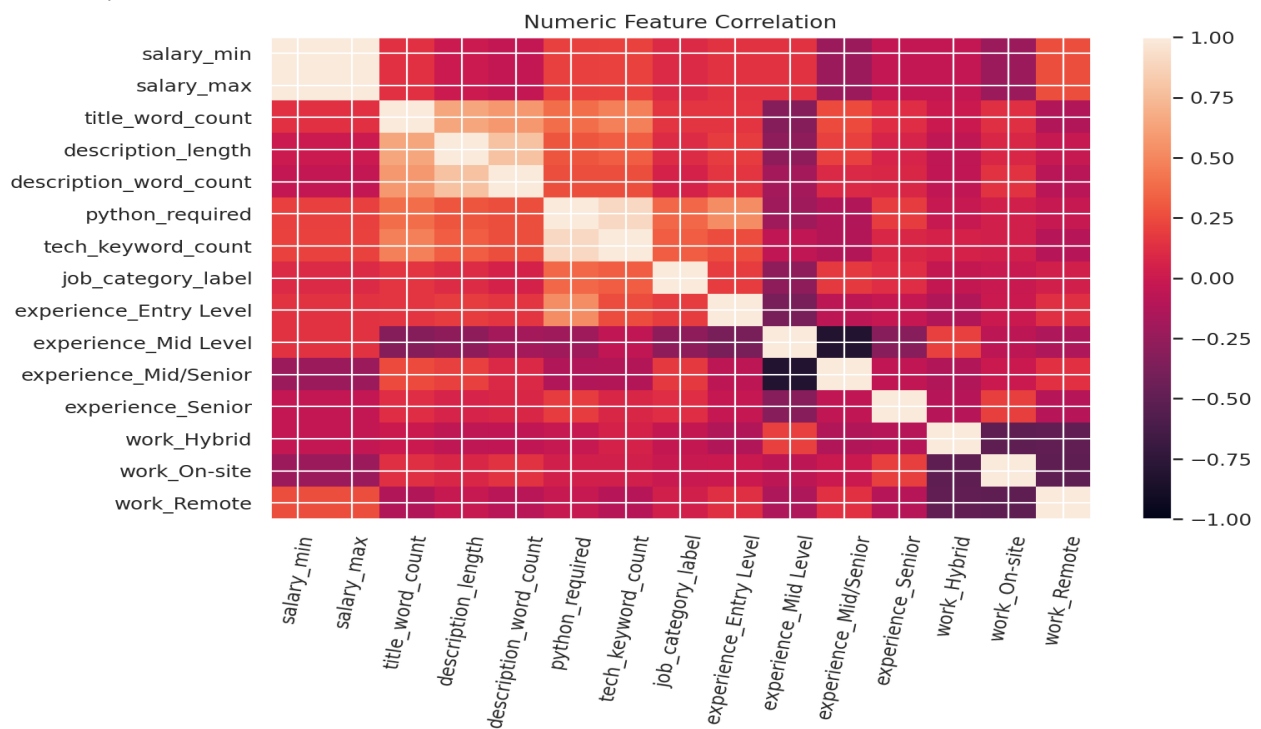


Figure 9. Correlation heatmap.

12. Streamlit Dashboard

The dashboard is implemented in `app.py` and contains Home, Dataset Viewer, Summary Statistics, Filtering & Search, Interactive Charts, Business Insights and Download pages. Sidebar controls allow users to filter by category, work mode and experience level, search text, and minimum salary. The dashboard displays KPIs for total jobs, companies, categories and average minimum salary.

The dashboard also provides CSV download functionality so users can export the processed dataset. To run it, use:
`streamlit run app.py`

13. Business Insights

The dashboard calculates decision-oriented observations from the processed dataset: the largest job category, most common work mode, category with the highest average minimum salary and the share of descriptions containing the Python keyword. These metrics demonstrate how an analytical dashboard can convert raw records into understandable business observations.

Because the source is a synthetic learning site, these observations describe the demonstration dataset only and should not be interpreted as real labor-market statistics.

14. Challenges & Solutions

Challenge: websites can change HTML structure. Solution: isolate scraping selectors in scraper.py and use exception handling.

Challenge: missing and duplicated records. Solution: explicit fill, type conversion and drop_duplicates steps.

Challenge: categorical variables cannot always be used directly in numerical analysis. Solution: label encoding and one-hot encoding.

Challenge: users need more than static charts. Solution: Streamlit filters, KPIs, interactive charts and download controls.

Challenge: the bundled project must run even before a fresh scrape. Solution: provide ready-made raw, cleaned and processed CSVs while keeping scraper.py available for refresh.

15. Conclusion

The completed project demonstrates an end-to-end Python analytics workflow from web scraping to an interactive dashboard. It covers the required programming, data collection, cleaning, feature engineering, statistical analysis, visualization and dashboard stages described in the supplied brief. The project can be extended with multiple sources, pagination handling, advanced search, additional dashboard filters and report export.

Recommended viva explanation: "I collected job listings with Requests and BeautifulSoup, cleaned and standardized them with Pandas, engineered analytical features including salary and job category, performed EDA using Matplotlib and Seaborn, and deployed the results through Streamlit with filters, KPIs, charts, insights and CSV download."